

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the AI-Powered Face Recognition System for Missing Persons. The system is designed to leverage deep learning-based facial recognition technology to assist law enforcement agencies, NGOs, and government bodies in identifying and locating missing individuals through real-time camera surveillance.

This document is intended for developers, testers, project managers, and academic evaluators. Version 1.0 covers the complete scope of the Minor Project for B.Tech VII Semester.

1.2 Document Conventions

The following conventions are used throughout this document:

- Bold text indicates section headings and key terms.
- REQ-XX identifiers uniquely label each functional requirement.
- High / Medium / Low indicate requirement priority levels.
- TBD indicates items to be determined in future iterations.

1.3 Intended Audience and Reading Suggestions

Audience	Relevant Sections
Developers	All sections, especially Sections 3, 4, and 5
Testers	Sections 4 (System Features) and 5 (Non-functional Requirements)
Project Guide / Evaluator	Sections 1, 2, and 6 (Project Plan)
End Users / Administrators	Sections 2 and 3 (Overall Description and Interfaces)

1.4 Product Scope

The AI-Powered Face Recognition System for Missing Persons is a web-based surveillance application that integrates real-time camera feeds with a deep learning face recognition engine (DeepFace — Facenet model) to automatically detect and alert when a registered missing person appears in the camera frame.

The system maintains a database of missing persons with their facial encodings and personal details. When a live camera detects a matching face, the system triggers an automated alert with a timestamped snapshot. Key objectives include:

- Reducing the time to identify missing individuals from days to seconds.
- Providing an accessible web interface for registration and alert management.
- Ensuring accurate, low-latency recognition using state-of-the-art AI models.
- Supporting simultaneous detection of multiple registered persons in a single frame.

2. Overall Description

2.1 Product Perspective

The system is a new, standalone product developed as a Minor Project for B.Tech VII Semester. It operates as a client-server application where a Python-based backend handles camera processing and face recognition, while a Flask web application provides the administrative front end. The system interfaces with a MySQL database for storing person records and alert logs.

The architecture consists of three major subsystems:

- Web Application Module: Flask-based registration, listing, and alert viewing portal.
- Camera Recognition Module: Real-time face detection and DeepFace-based matching engine.
- Database Module: MySQL database managing persons, encodings, and alert records.

2.2 Product Functions

The major functions of the system are:

- Missing Person Registration: Upload photo and personal details; system extracts and stores facial encoding.
- Real-Time Face Recognition: Continuously analyze camera frames and compare detected faces against the database.
- Automated Alert Generation: Save timestamped snapshot and log alert when a match is found above confidence threshold.
- Alert Dashboard: View all triggered alerts with person details, confidence scores, and snapshots.
- Missing Persons List: View, search, and manage registered missing persons.
- Multi-Face Simultaneous Detection: Independently track and identify multiple faces in a single frame.

2.3 User Classes and Characteristics

- System Administrator: Technical user responsible for running the camera recognition module, managing the database, and configuring system parameters.
- Operator / NGO Staff: Non-technical user who registers missing persons via the web interface and monitors the alerts dashboard.
- Law Enforcement Personnel: Supervisory user who reviews alerts and coordinates response actions.

2.4 Operating Environment

- Operating System: Windows 10/11
- Programming Language: Python 3.11+
- Web Framework: Flask 2.x

- Database: MySQL 8.x
- AI Library: DeepFace (Facenet model), OpenCV 4.x
- Camera: USB Webcam (minimum 720p resolution)
- Hardware: Minimum Intel Core i5, 8GB RAM, GPU optional but recommended for faster inference
- Browser: Any modern browser (Chrome, Firefox, Edge) for the web interface

2.5 Design and Implementation Constraints

- The system must use Python as the primary programming language.
- Face recognition must use the DeepFace library with the Facenet model for 128-dimensional embeddings.
- The database must be MySQL with the schema: missing_persons and alerts tables with referential integrity.
- The web interface must be built using Flask with HTML/CSS templates (Jinja2).
- Camera recognition runs as a separate process from the web server.
- Recognition threshold is fixed at cosine similarity ≥ 0.70 to balance accuracy and recall.

2.6 User Documentation

- User Manual: Step-by-step guide for registration, camera operation, and alert management.
- README.md: Installation and setup instructions including dependency installation commands.
- Inline Help: Contextual instructions on the web interface pages.

2.7 Assumptions and Dependencies

- A functional webcam is connected and accessible as VideoCapture(1) or VideoCapture(0).
- Internet connectivity is available during initial setup to download DeepFace model weights (~92MB).
- MySQL server is running locally with credentials root/admin.
- Registration photos must contain at least one clearly visible frontal face.
- The system assumes consistent indoor lighting for optimal recognition accuracy.

3. External Interface Requirements

3.1 User Interfaces

The web application provides the following pages:

- Home Page (home.html): Navigation hub with links to register, list, and alerts pages.

- Registration Page (register.html): Form to enter full name, age, gender, last seen location, contact info, and upload a photo. Submits via POST to /register.
- Missing Persons List (list.html): Table of all registered missing persons with status indicators.
- Alerts Page (alerts.html): Chronological list of triggered alerts showing person name, detection time, confidence score, camera location, and snapshot image.
- Camera Window: OpenCV-rendered window showing live feed with bounding boxes — green for matched persons (FOUND label with name and confidence), red for unknowns.

3.2 Hardware Interfaces

- USB Webcam: Accessed via OpenCV VideoCapture API. Minimum 640x480 resolution. Supports standard UVC protocol.
- CPU/GPU: DeepFace inference runs on CPU by default; GPU acceleration via CUDA is supported if available.
- Storage: Minimum 500MB free disk space for alert snapshots and model weights.

3.3 Software Interfaces

Component	Version	Purpose
Flask	2.x	Web framework for registration and alert portal
OpenCV	4.x	Camera access, face detection, image processing
DeepFace	Latest	Deep learning face encoding (Facenet model)
MySQL Connector	8.x	Database connectivity for person and alert records
Pickle	stdlib	Serialization of face encodings for DB storage
NumPy	1.x	Cosine similarity computation for face matching

3.4 Communications Interfaces

- The web application communicates over HTTP on localhost:5000 (Flask development server).
- Database communication uses the MySQL protocol on port 3306 (localhost).
- No external network communication is required during runtime (model weights pre-downloaded).

4. System Features

4.1 Missing Person Registration

4.1.1 Description and Priority

Allows authorized operators to register a missing person by providing their personal details and a facial photograph. The system extracts a 128-dimensional facial encoding using DeepFace and stores it in the database. Priority: HIGH.

4.1.2 Stimulus/Response Sequences

1. Operator navigates to /register page.
2. Operator fills in full name, age, gender, last seen location, contact details, and uploads a photo.
3. System processes the photo, detects the face, extracts the DeepFace encoding.
4. System stores the person record and encoding in the missing_persons table.
5. System displays a success message and redirects to the persons list.

4.1.3 Functional Requirements

REQ ID	Requirement	Priority
REQ-01	System shall accept JPEG/PNG images up to 10MB for registration.	High
REQ-02	System shall extract a 128-dim Facenet encoding from the uploaded photo.	High
REQ-03	System shall store the encoding as a serialized binary blob in MySQL.	High
REQ-04	System shall reject registration if no face is detected in the photo.	High
REQ-05	System shall store full name, age, gender, last seen location, and contact info.	High

4.2 Real-Time Face Recognition

4.2.1 Description and Priority

The camera module continuously reads frames from a webcam, detects faces using Haar Cascade, extracts DeepFace encodings, and compares them against all registered missing persons using cosine similarity. Priority: HIGH.

4.2.2 Stimulus/Response Sequences

6. Operator launches camera_recognition.py.
7. System loads all missing person encodings from the database.
8. For every 5th frame, system detects faces and crops each face region.
9. System extracts DeepFace encoding for each cropped face.
10. System computes cosine similarity against all registered encodings.
11. If similarity ≥ 0.70 , the face is labeled FOUND with the person's name and confidence %.
12. Otherwise, the face is labeled Unknown with a red bounding box.

4.2.3 Functional Requirements

REQ ID	Requirement	Priority
REQ-06	System shall process video frames at minimum 5 FPS on standard hardware.	High

REQ ID	Requirement	Priority
REQ-07	System shall detect multiple faces simultaneously in a single frame.	High
REQ-08	System shall use position-based tracking to maintain stable per-face identity across frames.	High
REQ-09	System shall reload encodings from DB every 100 frames to detect newly registered persons.	Medium
REQ-10	System shall display green bounding box and FOUND label for matched faces.	High
REQ-11	System shall display red bounding box and Unknown label for unmatched faces.	High

4.3 Automated Alert Generation

4.3.1 Description and Priority

When a matching face is detected above the confidence threshold, the system saves a timestamped snapshot and logs an alert record in the database. A 30-second cooldown prevents duplicate alerts for the same person. Priority: HIGH.

4.3.2 Stimulus/Response Sequences

13. Recognition engine confirms a match above 70% cosine similarity.
14. System checks if the last alert for this person was more than 30 seconds ago.
15. System saves a JPEG snapshot to the alerts/ directory.
16. System inserts an alert record into the alerts table with person ID, timestamp, confidence, camera location, and snapshot path.

4.3.3 Functional Requirements

REQ ID	Requirement	Priority
REQ-12	System shall save alert snapshots as JPEG files in the alerts/ directory.	High
REQ-13	System shall log person ID, timestamp, confidence score, and camera location in the alerts table.	High
REQ-14	System shall enforce a 30-second cooldown per person to prevent duplicate alerts.	Medium
REQ-15	System shall name snapshots with format: alert_{id}_{YYYYMMDD_HHMMSS}.jpg.	Low

4.4 Alert Dashboard

4.4.1 Description and Priority

The web interface provides a page to view all triggered alerts with details and snapshot previews. Priority: MEDIUM.

4.4.2 Functional Requirements

REQ ID	Requirement	Priority
REQ-16	System shall display all alerts sorted by detection time descending.	Medium
REQ-17	System shall display person name, confidence score, camera location, and snapshot for each alert.	Medium
REQ-18	System shall allow filtering alerts by person name.	Low

4.5 Missing Persons Management

4.5.1 Description and Priority

Operators can view the list of all registered missing persons. Priority: MEDIUM.

4.5.2 Functional Requirements

REQ ID	Requirement	Priority
REQ-19	System shall display all registered persons with their name, age, gender, status, and last seen location.	Medium
REQ-20	System shall show the registration date and photo thumbnail for each person.	Low

5. Nonfunctional Requirements

5.1 Performance Requirements

- The system shall process and display camera frames with a maximum end-to-end latency of 500ms on a standard laptop (Intel Core i5, 8GB RAM).
- Face detection shall complete within 50ms per frame using Haar Cascade.
- DeepFace encoding extraction shall complete within 300ms per face crop.
- Database queries for loading encodings shall return results within 1 second for up to 1000 registered persons.
- The web application shall respond to HTTP requests within 2 seconds.

5.2 Safety Requirements

- The system shall not store raw video footage; only alert snapshots are persisted.
- Alert snapshots shall be stored locally and not transmitted over any external network.
- The system shall not make any automated decisions beyond triggering an alert; all response actions remain with human operators.

5.3 Security Requirements

- The web application shall require authentication before accessing registration and alert pages (future enhancement, TBD).

- Database credentials shall be stored in a configuration file, not hardcoded in source code (future enhancement).
- Face encodings shall be stored as serialized binary blobs, not as raw images, to protect biometric data.
- The system shall comply with applicable data protection norms regarding storage of biometric information.

5.4 Software Quality Attributes

- **Reliability:** The camera module shall recover gracefully from frame read failures without crashing.
- **Usability:** The web interface shall be operable by non-technical users with minimal training.
- **Maintainability:** Source code shall follow modular design with separate files for FaceProcessor, camera recognition, and Flask app.
- **Portability:** The system shall run on both Windows and Linux operating systems.
- **Scalability:** The encoding comparison loop shall support up to 500 registered persons without performance degradation.

6. Project Plan

6.1 Team Members

S.No.	Name	Roll No.	Role
1.	Vishesh Mishra	TBD	Lead Developer

6.2 Division of Work

Phase	Tasks	Member
Analysis	Requirements gathering, system design, database schema	Vishesh Mishra
Design	Architectural design, UI wireframes, ER diagram	Vishesh Mishra
Coding	Flask app, face processor, camera recognition module	Vishesh Mishra
Testing	Unit testing, integration testing, performance benchmarking	Vishesh Mishra
Documentation	SRS, Synopsis, User Manual	Vishesh Mishra

6.3 Time Schedule

Phase	Start Date	End Date	Duration
Requirements & Analysis	Jan 2026	Jan 2026	2 Weeks
System Design	Feb 2026	Feb 2026	2 Weeks
Development & Coding	Feb 2026	Mar 2026	4 Weeks
Testing & Debugging	Mar 2026	Mar 2026	2 Weeks
Documentation	Apr 2026	Apr 2026	1 Week
Final Review & Submission	Apr 2026	Apr 2026	1 Week

Appendix A: Glossary

Term / Acronym	Definition
SRS	Software Requirements Specification
AI	Artificial Intelligence
DeepFace	Open-source deep learning facial recognition library by Meta Research
Facenet	Deep neural network architecture producing 128-dimensional face embeddings
Cosine Similarity	Metric measuring the cosine of the angle between two embedding vectors; ranges from -1 to 1
Haar Cascade	Machine learning-based object detection algorithm used for fast face detection
Flask	Lightweight Python web framework used for building the administrative portal
Encoding	128-dimensional numerical vector representing unique facial features
Bounding Box	Rectangle drawn around a detected face in the camera frame
FPS	Frames Per Second — measure of video processing speed
TBD	To Be Determined
REQ	Requirement identifier prefix used in this document

Appendix B: To Be Determined List

- TBD-01: Authentication mechanism for web application login.
- TBD-02: External notification system (SMS/email) for alert delivery.
- TBD-03: Multi-camera support for networked CCTV feeds.
- TBD-04: Cloud database deployment for centralized access.
- TBD-05: Mobile application interface for field officers.